

Rapport : Résolution du Problème NP-Difficile de Kemeny.

Goales Saphiya

Teibi Zahra

Introduction

La théorie du vote s'intéresse aux règles qui permettent de trouver une décision collective à partir des préférences individuelles. Parmi les différentes méthodes existantes, l'agrégation de classement constitue un problème fondamental au cœur de la théorie du choix social, avec des applications allant de l'apprentissage automatique à la recherche web [1].

Parmi les règles de vote, la règle de Kemeny établit un classement de consensus qui minimise la distance totale de Kendall-tau (le nombre total de désaccords (paires inversées) entre le consensus et le profil de préférences). Le profil de préférences est l'ensemble $P = \{P_1, P_2, \dots, P_n\}$, où chaque P_i est le classement donné par le votant i .

D'un point de vue théorique, la règle de Kemeny possède des propriétés robustes, telles que le fait de satisfaire à la fois :

- le critère de Condorcet, c'est à dire que si un candidat, confronté à chaque autre candidat, est toujours gagnant (vainqueur de Condorcet), alors il sera classé premier dans le classement de Kemeny [2, 3].
- l'axiome de renforcement, qui stipule que si l'on considère deux groupes distincts de votants ayant les mêmes résultats, alors ce même résultat doit être obtenu lorsque l'on combine les votes des deux groupes [3].
- la neutralité qui garantit que si deux candidats échangent leurs positions dans les préférences de tous les votants, ils doivent également échanger leurs positions dans le classement final. Ainsi, tous les candidats sont traités de manière égale [3].

Problématique : cette solidité théorique est confrontée à une difficulté pratique majeure : sa complexité. En effet, le calcul d'un classement de Kemeny optimal est un problème NP-difficile [4, 5]. Ce résultat est valable même lorsque le nombre de votants est petit, pour $n = 4$, par exemple [5].

Afin de contrer cette problématique, des méthodes de décompositions ont été proposées. Ces méthodes réduisent l'instance originale à de plus petits sous-problèmes indépendants qu'il faudra ensuite résoudre.

Ce projet vise à implémenter des approches permettant de résoudre le problème de Kemeny de manière exacte et efficace, même sur des instances de grande taille. Ainsi l'objectif est de comparer les méthodes de résolution (programmation dynamique et programmation linéaire) ainsi que les méthodes de décomposition (le critère de Condorcet étendu [6] et la règle de majorité des 3/4 [4]). Afin d'obtenir des résultats d'évaluation réalistes, nous évaluerons ces méthodes sur des instances réelles stockées dans la bibliothèque Preflib. De plus, ce projet se concentre principalement sur des préférences complètes et sans égalité.

Table des matières

1	Implémentation	3
1.1	Réduction de données	3
1.1.1	La règle de majorité des 3/4	3
1.1.2	Le critère de Condorcet étendu	3
1.2	Résolution des sous instances	4
1.2.1	Programmation Dynamique	4
1.2.2	Programmation linéaire	4
2	Analyse des résultats	4
3	(Bonus) Génération d'instances	4
3.1	implémentation	4
3.2	analyse des résultats	4
4	Conclusion	4

1 Implémentation

Nous avons développé ce projet en python. Tout le code décrit dans cette section est stocké dans le fichier `instance.py`.

Nous avons défini une classe *Instance* afin d'y stocker l'instance (une élection) sur laquelle nous évaluons nos méthodes. Une instance représente un profil de préférences sur un ensemble de candidats.

Pour cette partie, nos instances sont issues de la bibliothèque *preflibtools*, stockées dans des fichiers `.soc`. Nous avons donc récupéré toutes les instances sur le github PrefLib : <https://github.com/PrefLib/PrefLib-Data/tree/main/datasets>

Ainsi, la classe *Instance* contient une méthode de lecture de ces fichiers. Pour ce faire, nous utilisons les méthodes prédéfinies dans la bibliothèque *preflibtools.instances*. Ensuite, on construit la matrice de préférences de cette instance à l'aide de la méthode *comptage*. Dans cette matrice, on associe la case (i,j) au nombre de votants préférant le candidat i au candidat j .

Pour résoudre les instances nous avons implémenté toutes les méthodes de résolutions et de composition évoquées.

1.1 Réduction de données

La réduction de données vise à identifier et à fixer des relations d'ordre entre candidats qui sont garanties d'apparaître dans toute solution optimale. Cette réduction constitue une phase de prétraitement essentielle pour rendre la résolution d'instances de grande taille abordable.

1.1.1 La règle de majorité des 3/4

1.1.2 Le critère de Condorcet étendu

Le critère de Condorcet étendu [6] constitue une version plus robuste du critère de Condorcet, qui se concentre sur un unique vainqueur.

Le principe est le suivant : si un sous-ensemble de candidat C_α est majoritairement préféré à tous les candidats de l'ensemble complémentaire $C_\beta = C \setminus C_\alpha$ alors pour tout $x \in C_\alpha$ et $y \in C_\beta$, x est préféré à y par plus de la moitié des électeurs. Ainsi cet ensemble C_α doit se placer en tête de tout classement optimal. Ce principe peut être généralisé avec plusieurs sous-ensembles formant une partition de l'ensemble des candidats C . Le classement de Kemeny sur C est obtenu en juxtaposant les classements de Kemeny trouvés sur les sous-ensembles d'une partition.

L'algorithme du critère de Condorcet étendu consiste à construire le graphe de majorité stricte, puis d'ordonner les composantes fortement connexes. Un graphe de majorité strict est un graphe dont les noeuds correspondent aux candidats, et un arc entre deux candidats a et b indique que le candidat a est préféré par plus de 50% des votants au candidat b .

Afin d'implémenter cet algorithme, nous avons utilisé une classe Graphe interne à la classe *Instance*. Un graphe est défini par une liste de noeuds et un dictionnaire d'adjacence. Cette classe permettra par la suite de créer le graphe de majorité d'une instance. Ce graphe peut être affichée grâce à la fonction *afficher_graphe*, qui utilise la bibliothèque *networkx*; ce qui nous permet de vérifier les résultats visuellement.

Pour identifier et ordonner les composantes fortement connexes, nous avons implémenté l'algorithme de Tarjan dans la fonction *condorcet_etendu* définie en dehors de la classe *Instance*. Une composante fortement connexe de taille > 1 représente un groupe incohérent, soit une sous-instance à résoudre. Par conséquent, pour chacune d'entre elles, on crée un objet de la classe *Instance*.

1.2 Résolution des sous instances

1.2.1 Programmation Dynamique

1.2.2 Programmation linéaire

2 Analyse des résultats

3 (Bonus) Génération d'instances

3.1 implémentation

3.2 analyse des résultats

4 Conclusion

Références

- [1] Dwork Cynthia, Kumar Ravi, Moni Naor, and D. Sivakumar. Rank aggregation methods for the web. 2001.
- [2] Davenport Andrew and Kalagnanam Jayant. A computational study of the kemeny rule for preference aggregation. 2004.
- [3] Young H. P. and Levenglick A. A consistent extension of condorcet's election principle. 1978.
- [4] Nadja Betzler, Robert Bredereck, and Rolf Niedermeier. Theoretical and empirical evaluation of data reduction for exact kemeny rank aggregation. 2013.
- [5] Fischer Felix, Hudry Olivier, and Niedermeier Rolf. Weighted tournament solutions. 2016.
- [6] Truchon Michel. An extension of the condorcet criterion and kemeny orders. Technical report, 1999.